

Presentation Question Catalog â Part 1 (LCD / monitor)

For: the LCD driver owner

Also covers: main.c and the whole project

Answers: not in this file. Write them separately; we will check them later.

Practice out loud. Be ready for a follow-up â why?â .

Project and 'main.c'

1. In one minute: what does the device do after power-on, and what are the three presentation scenarios (SW1 live, live refresh, 30 s sleep + wake restore)?
2. LCD and VL53L1X share one bus. Which bus, which pins (SCL/SDA), and why is sharing a constraint?
3. Name the application states. What does one SW1 press do in Ready, Live, Hold, and Sleep?
4. What does Hold freeze â the last millimetre I2C sample, or the centimetre value in the application? Why does that matter?
5. After 30 s idle: what is stopped, what is turned off, and what is remembered so wake can restore Hold?
6. Potentiometer bands (0â 33 / 34â 66 / 67â 100 %): what does each mode do? Does SW1 still work in all three?
7. LCD DEBUG: what is on line 0 and line 1, and why is the LCD written on Enter, not on every key?
8. Why is USART2 RX interrupt-driven (ring buffer) instead of polling HAL_UART_Receive in a loop that may sleep 100 ms?
9. TIM7 replaced SysTick as the tick source. Why do we still call HAL_GetTick()? Where does the energy saving actually come from (coarse sleep + EXTI in ST_SLEEP)?
10. In ST_SLEEP we suspend the tick and __WFI(). Is this STM32 Stop/Standby or Sleep, and why (RAM, Hold, LCD)? What is the intended wake source?

LCD driver (your part)

11. HD44780 16^ 2, 4-bit mode, via a PCF8574 backpack. Map P0â P7 (RS, RW, EN, BL, data). Which 7-bit I2C addresses do we probe, in which order?
12. LCD_Init(hi2c): argument, return type (void), probe + 4-bit boot nibbles (0x30 ^ 3, 0x20), then which setup commands? What if no backpack ACKs?
13. Why must MX_I2C1_Init() run first? Open-drain + pull-ups on PB6/PB7: what must CubeMX generate?
14. LCD_WriteLine(row, s): why always exactly 16 characters? How is the row chosen (DDRAM 0x80 / 0xC0)?
15. For one character, roughly how does one nibble leave the MCU (expander byte, EN high, EN low)? Why a ~µs busy-wait instead of HAL_Delay for EN?
16. LCD_Sleep vs LCD_Wake: display/backlight, is DDRAM lost? Who redraws the text after wake?
17. Update_LCD_Standard(dist): unit of dist, which row, which string? Does current main still call Update_LCD_Debug?
18. Why is I2C DMA a poor fit for this backpack? Why is EXTI0 (PB0 / ToF INT) disabled during LCD writes?

19. In STANDARD Live mode, who writes line 0 (cm) and who writes the "On: Ns" countdown?
In LCD DEBUG, why echo the typing line on the VCP only?

20. Trap: "Hold showed 200 cm at ~4.5 m" is that the LCD %3d format? What actually limited it (CM_MAX), and what is the LCD's real digit limit?

No answers here. Send yours when you want them checked.*